ECE 759
High Performance Computing for Engineering Applications
Assignment 4

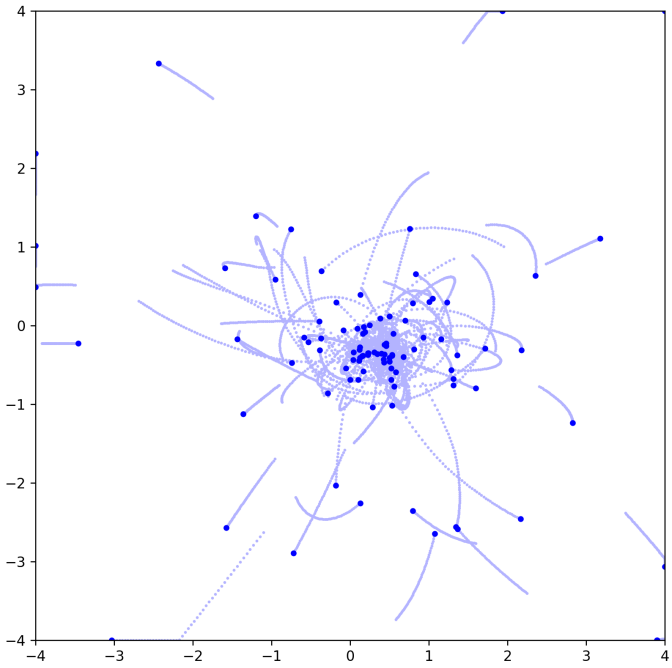
Submitted by: Zhengyang Wan

My GitHub link: https://github.com/BillWan-zzzyyy/repo759/tree/main/HW04

1.

Run the `nbody.py` script either on your personal machine or on the Euler compute node. If you run it locally, you will see an animation of the N-body simulation. At the end of the simulation, the code will automatically save a plot. Rename this plot as `task1.png` and submit it to Canvas.

Answer:



2.

(a) Write a program `task2.cpp` that accomplishes the following:

- Complete the provided skeleton code from `task2.cpp`.

- Compile: `g++ task2.cpp -Wall -O3 -std=c++17 -o task2`
- Run the program; make sure you use Slurm:
  `./task2 number_of_particles simulation_end_time`
- (Optional) For debugging and visualization, we provide a function `savePositions ToCSV`. This function saves the positions at each iteration to a CSV file. You can then use `plot_positions.py` to visualize the C++ simulation results. However, if you're measuring performance, disable this function as it introduces significant I/O overhead.

Answer:

```
#include <cstddef>
#include <iostream>
#include <fstream>
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <random>

using std::chrono::high_resolution_clock;
using std::chrono::duration;

// Constants
const double G = 1.0;          // Gravitational constant
const double softening = 0.1;  // Softening length
const double dt = 0.01;        // Time step
const double board_size = 4.0; // Size of the board

// Function to calculate acceleration due to gravity
void getAcc(const double pos[][3], const double mass[], double acc[][3], int N) {
    for (int i = 0; i < N; i++) {
        acc[i][0] = 0.0;
        acc[i][1] = 0.0;
        acc[i][2] = 0.0;

        // sum the pull from every other particle (same as getAcc in nbody.py)
        for (int j = 0; j < N; j++) {
            if (i != j) {
                double dx = pos[j][0] - pos[i][0];
                double dy = pos[j][1] - pos[i][1];
                double dz = pos[j][2] - pos[i][2];
                double inv_r3 = std::pow(dx * dx + dy * dy + dz * dz + softening *
    softening, -1.5);
                acc[i][0] += G * (dx * inv_r3) * mass[j];
                acc[i][1] += G * (dy * inv_r3) * mass[j];
                acc[i][2] += G * (dz * inv_r3) * mass[j];
            }
        }
```

```
    }
}

// For debug: save positions to a CSV file
void savePositionsToCSV(const double pos[][3], int N, int step, const std::string &
    filename) {
    std::ofstream file;

    // Open the file in append mode
    file.open(filename, std::ios_base::app);

    if (file.is_open()) {
        file << step << ",[";
        for (int i = 0; i < N; i++) {
            if (i != N - 1)
                file << "[" << pos[i][0] << "," << pos[i][1] << "," << pos[i][2] <<
    "],";
            else
                file << "[" << pos[i][0] << "," << pos[i][1] << "," << pos[i][2] <<
    "]";
        }
        file << "]\n";  // Newline for separation between steps
        file.close();
    } else {
        std::cerr << "Unable to open file!" << std::endl;
    }
}

int main(int argc, char *argv[]) {
    high_resolution_clock::time_point start;
    high_resolution_clock::time_point end;
    duration<double, std::milli> duration_sec;
    start = high_resolution_clock::now();

    // Check if correct number of arguments are provided
    if (argc != 3) {
        std::cerr << "Usage: " << argv[0] << " <number_of_particles> <
    simulation_end_time>" << std::endl;
        return 1;
    }

    // Read N and tEnd from command line
    int N = std::stoi(argv[1]);     // Number of particles
    double tEnd = std::stod(argv[2]); // Time at which simulation ends

    // File to save positions
    std::string filename = "positions.csv";

    // Clear the file before starting simulation (optional)
    std::ofstream file;
    file.open(filename, std::ofstream::out | std::ofstream::trunc);
```

```
    file.close();

    // Allocate dynamic arrays based on N
    double* mass = new double[N];
    double(*pos)[3] = new double[N][3];
    double(*vel)[3] = new double[N][3];
    double(*acc)[3] = new double[N][3];

    // Create a random number engine
    std::mt19937 generator(std::random_device{}());

    // Create random distributions
    std::uniform_real_distribution<double> uniform_dist(0.0, 1.0);
    std::normal_distribution<double> normal_dist(0.0, 1.0);

    // Simulation parameters
    double t = 0.0;

    // Set initial masses and random positions/velocities
    for (int i = 0; i < N; i++) {
        mass[i] = uniform_dist(generator);

        pos[i][0] = normal_dist(generator);
        pos[i][1] = normal_dist(generator);
        pos[i][2] = normal_dist(generator);

        vel[i][0] = normal_dist(generator);
        vel[i][1] = normal_dist(generator);
        vel[i][2] = normal_dist(generator);
    }

    // Convert to Center-of-Mass frame
    double velCM[3] = {0.0, 0.0, 0.0};
    double totalMass = 0.0;
    for (int i = 0; i < N; i++) {
        velCM[0] += vel[i][0] * mass[i];
        velCM[1] += vel[i][1] * mass[i];
        velCM[2] += vel[i][2] * mass[i];
        totalMass += mass[i];
    }

    velCM[0] /= totalMass;
    velCM[1] /= totalMass;
    velCM[2] /= totalMass;

    for (int i = 0; i < N; i++) {
        vel[i][0] -= velCM[0];
        vel[i][1] -= velCM[1];
        vel[i][2] -= velCM[2];
    }
```

```
    // Initial accelerations
    getAcc(pos, mass, acc, N);

    // Number of timesteps
    int Nt = int(tEnd / dt);

    // Main simulation loop
    for (int step = 0; step < Nt; step++) {

        // (1/2) kick
        for (int i = 0; i < N; i++) {
            vel[i][0] += acc[i][0] * dt / 2.0;
            vel[i][1] += acc[i][1] * dt / 2.0;
            vel[i][2] += acc[i][2] * dt / 2.0;
        }

        // Drift
        for (int i = 0; i < N; i++) {
            pos[i][0] += vel[i][0] * dt;
            pos[i][1] += vel[i][1] * dt;
            pos[i][2] += vel[i][2] * dt;
        }

        // Ensure particles stay within the board limits
        for (int i = 0; i < N; i++) {
            for (int k = 0; k < 3; k++) {
                if (pos[i][k] > board_size)
                    pos[i][k] = board_size;
                if (pos[i][k] < -board_size)
                    pos[i][k] = -board_size;
            }
        }

        // Update accelerations
        getAcc(pos, mass, acc, N);

        // (1/2) kick
        for (int i = 0; i < N; i++) {
            vel[i][0] += acc[i][0] * dt / 2.0;
            vel[i][1] += acc[i][1] * dt / 2.0;
            vel[i][2] += acc[i][2] * dt / 2.0;
        }

        // Update time
        t += dt;

        // For debug: save positions to CSV at each step
        // savePositionsToCSV(pos, N, step, filename);
    }

    // Clean up dynamically allocated memory
```

```
    delete[] mass;
    delete[] pos;
    delete[] vel;
    delete[] acc;

    end = high_resolution_clock::now();
    duration_sec = std::chrono::duration_cast<duration<double, std::milli> >(end -
    start);
    std::cout << "time: " << duration_sec.count() << "ms\n";

    return 0;
}
```

3.

(a) Write a program `task3.cpp` that accomplishes the following:

- Add an argument `num_threads`, which specifies the number of threads to launch with OpenMP.
- Insert OpenMP directives (`#pragma omp parallel for`) in sections of the code that can be parallelized.
- Compile: `g++ task3.cpp -Wall -O3 -std=c++17 -o task3 -fopenmp`
- Run the program; make sure you use Slurm and request multiple CPU cores: `./task3 number_of_particles simulation_end_time num_threads`

Answer:

```
#include <cstddef>
#include <iostream>
#include <fstream>
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <random>
#include <string>

#include <omp.h>

using std::chrono::high_resolution_clock;
using std::chrono::duration;

// Constants
const double G = 1.0;          // Gravitational constant
const double softening = 0.1;  // Softening length
```

```
const double dt = 0.01;        // Time step
const double board_size = 4.0; // Size of the board

// Function to calculate acceleration due to gravity
void getAcc(const double pos[][3], const double mass[], double acc[][3], int N) {
    // each i only writes acc[i], so the rows can be split between threads
    #pragma omp parallel for schedule(runtime)
    for (int i = 0; i < N; i++) {
        acc[i][0] = 0.0;
        acc[i][1] = 0.0;
        acc[i][2] = 0.0;

        // sum the pull from every other particle (same as getAcc in nbody.py)
        for (int j = 0; j < N; j++) {
            if (i != j) {
                double dx = pos[j][0] - pos[i][0];
                double dy = pos[j][1] - pos[i][1];
                double dz = pos[j][2] - pos[i][2];
                double inv_r3 = std::pow(dx * dx + dy * dy + dz * dz + softening *
    softening, -1.5);
                acc[i][0] += G * (dx * inv_r3) * mass[j];
                acc[i][1] += G * (dy * inv_r3) * mass[j];
                acc[i][2] += G * (dz * inv_r3) * mass[j];
            }
        }
    }
}

// For debug: save positions to a CSV file
void savePositionsToCSV(const double pos[][3], int N, int step, const std::string &
    filename) {
    std::ofstream file;

    // Open the file in append mode
    file.open(filename, std::ios_base::app);

    if (file.is_open()) {
        file << step << ",[";
        for (int i = 0; i < N; i++) {
            if (i != N - 1)
                file << "[" << pos[i][0] << "," << pos[i][1] << "," << pos[i][2] <<
    "],";
            else
                file << "[" << pos[i][0] << "," << pos[i][1] << "," << pos[i][2] <<
    "]";
        }
        file << "]\n";  // Newline for separation between steps
        file.close();
    } else {
        std::cerr << "Unable to open file!" << std::endl;
    }
```

```
}

int main(int argc, char *argv[]) {
    high_resolution_clock::time_point start;
    high_resolution_clock::time_point end;
    duration<double, std::milli> duration_sec;
    start = high_resolution_clock::now();

    // Check if correct number of arguments are provided
    if (argc != 4 && argc != 5) {
        std::cerr << "Usage: " << argv[0]
                  << " <number_of_particles> <simulation_end_time> <num_threads> [
    static|dynamic|guided]" << std::endl;
        return 1;
    }

    // Read N, tEnd and num_threads from command line
    int N = std::stoi(argv[1]);     // Number of particles
    double tEnd = std::stod(argv[2]); // Time at which simulation ends
    int num_threads = std::stoi(argv[3]); // Number of OpenMP threads
    if (num_threads < 1) {
        std::cerr << "num_threads must be at least 1" << std::endl;
        return 1;
    }

    // Optional scheduling policy for Problem 4, static by default.
    // All loops use schedule(runtime), so this choice is used by every loop.
    omp_sched_t kind = omp_sched_static;
    if (argc == 5) {
        std::string policy = argv[4];
        if (policy == "static") {
            kind = omp_sched_static;
        } else if (policy == "dynamic") {
            kind = omp_sched_dynamic;
        } else if (policy == "guided") {
            kind = omp_sched_guided;
        } else {
            std::cerr << "policy must be static, dynamic or guided" << std::endl;
            return 1;
        }
    }
    omp_set_num_threads(num_threads);
    // chunk size 0 means the default chunk size of that policy
    omp_set_schedule(kind, 0);

    // File to save positions
    std::string filename = "positions.csv";

    // Clear the file before starting simulation (optional)
    std::ofstream file;
    file.open(filename, std::ofstream::out | std::ofstream::trunc);
```

```
    file.close();

    // Allocate dynamic arrays based on N
    double* mass = new double[N];
    double(*pos)[3] = new double[N][3];
    double(*vel)[3] = new double[N][3];
    double(*acc)[3] = new double[N][3];

    // Create a random number engine
    std::mt19937 generator(std::random_device{}());

    // Create random distributions
    std::uniform_real_distribution<double> uniform_dist(0.0, 1.0);
    std::normal_distribution<double> normal_dist(0.0, 1.0);

    // Simulation parameters
    double t = 0.0;

    // Set initial masses and random positions/velocities
    for (int i = 0; i < N; i++) {
        mass[i] = uniform_dist(generator);

        pos[i][0] = normal_dist(generator);
        pos[i][1] = normal_dist(generator);
        pos[i][2] = normal_dist(generator);

        vel[i][0] = normal_dist(generator);
        vel[i][1] = normal_dist(generator);
        vel[i][2] = normal_dist(generator);
    }

    // Convert to Center-of-Mass frame
    double velCM[3] = {0.0, 0.0, 0.0};
    double totalMass = 0.0;
    for (int i = 0; i < N; i++) {
        velCM[0] += vel[i][0] * mass[i];
        velCM[1] += vel[i][1] * mass[i];
        velCM[2] += vel[i][2] * mass[i];
        totalMass += mass[i];
    }

    velCM[0] /= totalMass;
    velCM[1] /= totalMass;
    velCM[2] /= totalMass;

    for (int i = 0; i < N; i++) {
        vel[i][0] -= velCM[0];
        vel[i][1] -= velCM[1];
        vel[i][2] -= velCM[2];
    }
```

```
    // Initial accelerations
    getAcc(pos, mass, acc, N);

    // Number of timesteps
    int Nt = int(tEnd / dt);

    // Main simulation loop
    for (int step = 0; step < Nt; step++) {

        // (1/2) kick
        #pragma omp parallel for schedule(runtime)
        for (int i = 0; i < N; i++) {
            vel[i][0] += acc[i][0] * dt / 2.0;
            vel[i][1] += acc[i][1] * dt / 2.0;
            vel[i][2] += acc[i][2] * dt / 2.0;
        }

        // Drift
        #pragma omp parallel for schedule(runtime)
        for (int i = 0; i < N; i++) {
            pos[i][0] += vel[i][0] * dt;
            pos[i][1] += vel[i][1] * dt;
            pos[i][2] += vel[i][2] * dt;
        }

        // Ensure particles stay within the board limits
        #pragma omp parallel for schedule(runtime)
        for (int i = 0; i < N; i++) {
            for (int k = 0; k < 3; k++) {
                if (pos[i][k] > board_size)
                    pos[i][k] = board_size;
                if (pos[i][k] < -board_size)
                    pos[i][k] = -board_size;
            }
        }

        // Update accelerations
        getAcc(pos, mass, acc, N);

        // (1/2) kick
        #pragma omp parallel for schedule(runtime)
        for (int i = 0; i < N; i++) {
            vel[i][0] += acc[i][0] * dt / 2.0;
            vel[i][1] += acc[i][1] * dt / 2.0;
            vel[i][2] += acc[i][2] * dt / 2.0;
        }

        // Update time
        t += dt;

        // For debug: save positions to CSV at each step
```

```
        // savePositionsToCSV(pos, N, step, filename);
    }

    // Clean up dynamically allocated memory
    delete[] mass;
    delete[] pos;
    delete[] vel;
    delete[] acc;

    end = high_resolution_clock::now();
    duration_sec = std::chrono::duration_cast<duration<double, std::milli> >(end -
    start);
    std::cout << "time: " << duration_sec.count() << "ms\n";

    return 0;
}
```

4.

(a) In `task3.cpp`, try the following scheduling policies: `static`, `dynamic`, and `guided`.

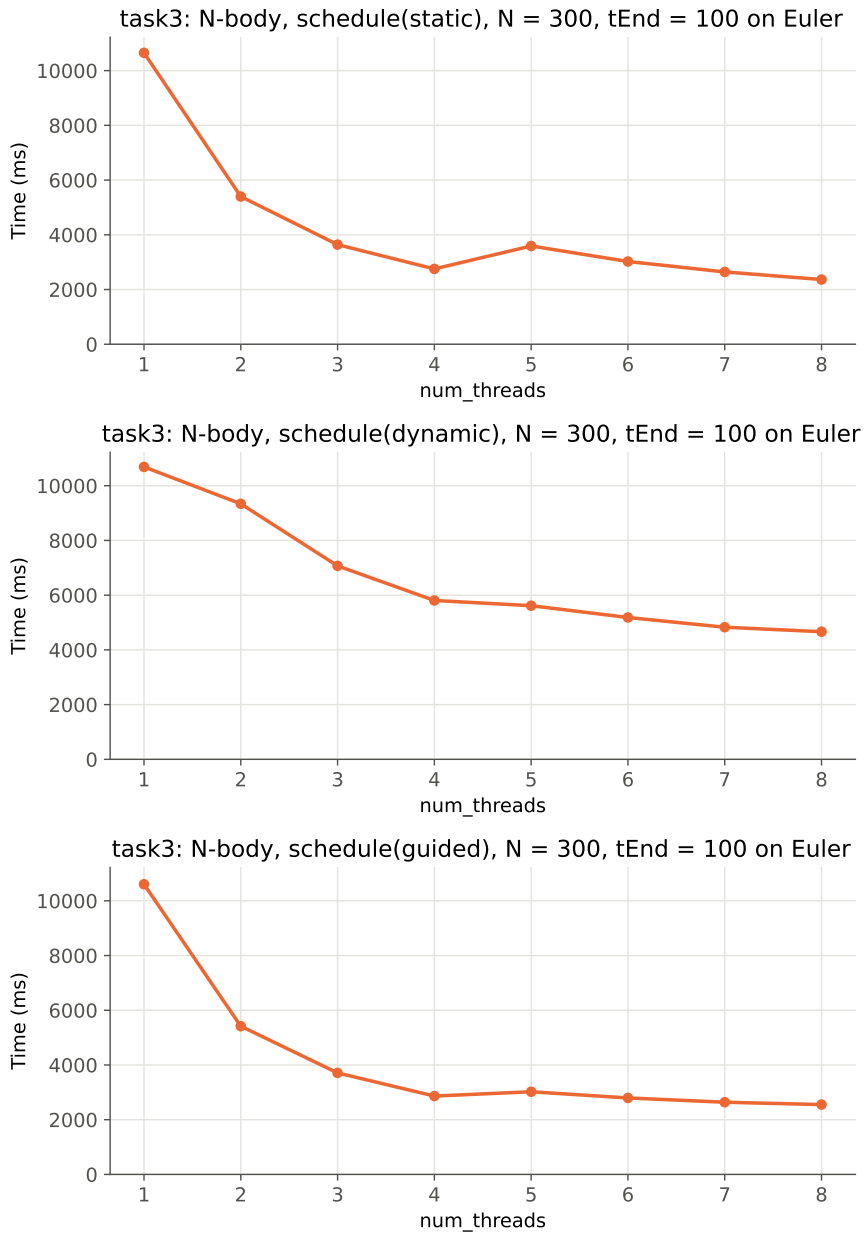
Answer: All `parallel for` loops in `task3.cpp` use `schedule(runtime)`, and the policy is picked with an optional fourth argument, e.g. `./task3 300 100 8 dynamic`. The program calls `omp_set_schedule` with `omp_sched_static`, `omp_sched_dynamic` or `omp_sched_guided` (default chunk size). Without the fourth argument `static` is used, so `./task3 number_of_particles simulation_end_time num_threads` still works as required.

(b) On Euler, via Slurm do the following: for each scheduling policy, run `task3` with the following parameters:

- `number_of_particles` = 100 or larger
- `simulation_end_time` = 100 or larger
- `num_threads` = $1, 2, \cdots, 8$

Generate plots called `task4.pdf` that show the time taken by the N-body simulation versus `num_threads` on a linear-linear scale. The `task4.pdf` should contain three plots, one for each scheduling policy.

Answer: The plots are submitted separately as `task4.pdf`. 300 particles and a simulation end time of 100 (10000 time steps) are used, i.e. `./task3 300 100 t policy`, with 8 cores requested from Slurm.



With one thread all three policies take about 10.6 s, the same as the serial `task2`. `static` scales best: 2.36 s at 8 threads, a speedup of about 4.5×. `guided` is close to it (2.55 s, about 4.2×). `dynamic` is much slower (4.67 s, only about 2.3×). With the default chunk size of 1, every single iteration is handed out through the shared work queue. The iterations here are very short: the kick and drift loops do only a few operations per particle, and even one row of `getAcc` is only 300 interactions. So the scheduling overhead is large compared to the work. Every particle costs the same amount of work, so `static` already balances the load and has the lowest overhead. `guided` hands out big chunks first and only gets small near the end, so it behaves almost like `static`. For `static` and `guided` the speedup flattens after about 4 threads, because the problem is

small and there are five parallel regions in every time step, each with its own fork/join and barrier cost. Each configuration was run only once on a shared node, so small bumps such as `static` at 5 threads are most likely timing noise.